

Case Study

High-Performance Financial Transaction System

Problem Statement

You are a Java Developer at a leading financial institution tasked with building a high-performance, scalable money transfer system that allows clients to view their account lists and transfer funds to other clients. The system must handle high concurrency, ensure data consistency, and integrate with multiple external services (e.g., fraud detection, notification services, ledger updates).

Expected Technology Stack

- Java / Spring Boot microservices
- Kafka for async messaging
- Redis for caching
- PostgreSQL for persistence
- Circuit breaker, retry, and idempotency

Key Requirements

1. Account Management

- Fetch and display a list of accounts for a given client.
- Account data must be cached for performance.

2. Money Transfer

- Support high-frequency transactions with low latency.
- Ensure atomicity (transactions succeed or fail completely).
- Handle retries in case of temporary failures (network issues, external service unavailability).

3. External Service Integration

- Fraud detection service (sync call, must be fast).
- Notification service (async, eventual consistency).
- Ledger service (must be eventually consistent).

4. Performance & Scalability

- Optimize for high throughput (thousands of transactions per second).
- Use Kafka for async processing where possible.
- Implement caching (Redis) to reduce database load.

5. Resilience

- Retry policies for transient failures (exponential backoff).
- Circuit breakers to avoid cascading failures.
- Idempotency to prevent duplicate transactions.

Evaluation Criteria

Correctness

No double-spending, data consistency.

Performance

Handles 1,000+ TPS with low latency.

Resilience

Retries, circuit breakers, idempotency.

Scalability

Microservices, Kafka, caching.

Code Quality

Clean architecture, tests, observability.

Bonus Challenges (Optional)

1. How would you handle a partial failure where money is debited but not credited?
2. How would you scale this globally (multi-region deployments)?
3. How would you introduce rate limiting to prevent abuse?